

Swagger 성공 (200)

Server response

Code	Details
200	Response body <pre>{ "success": true, "label": "궁정", "confidence": 0.9995, "raw_label": "1", "model_name": "daekeun-ml/koelectra-small-v3-nsmc", "user": "사용자A", "input_text": "배송도 빠르고 제품 품질도 좋아서 정말 만족합니다." }</pre> Response headers <pre>content-length: 225 content-type: application/json date: Tue, 07 Apr 2026 02:16:32 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
  "success": true,
  "label": "string",
  "confidence": 0,
  "raw_label": "string",
  "model_name": "string",
  "user": "string",
  "input_text": "string"
}
```

API Key 없이 401

Curl

```
curl -X 'POST' \
  'http://localhost:8000/predict' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "review": "배송도 빠르고 제품 품질도 좋아서 정말 만족합니다."
  }'
```

Request URL

http://localhost:8000/predict

Server response

Code	Details
401	Error: Unauthorized Response body <pre>{ "detail": "API Key가 필요합니다. X-API-Key 헤더를 포함해 주세요." }</pre> Response headers <pre>content-length: 81 content-type: application/json date: Tue, 07 Apr 2026 02:15:51 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

Streamlit 화면

설정

API Key

test-key-001

테스트용 키 예시: test-key-001

한국어 영화/상품 후기 감정분석기

리뷰를 입력하면 긍정 / 부정과 신뢰도를 반환합니다.

리뷰 입력

```
{
  "review": "배송도 빠르고 제품 품질도 좋아서 정말 만족합니다."
}
```

감정 분석 실행

분석 완료

한국어 감정분석기

Korean Review Sentiment Analysis API - Swagger UI

설정

API Key

test-key-001

테스트용 키 예시: test-key-001

감정 분석 실행

분석 완료

감정 결과

1

신뢰도

0.9994

상세 정보

```
{
  "success": true
  "label": "1"
  "confidence": 0.9994
  "raw_label": "1"
  "model_name": "daekeun-ml/koelectra-small-v3-nsmc"
  "user": "사용자A"
  "input_text": "{
    \"review\": \"배송도 빠르고 제품 품질도 좋아서 정말 만족합니다.\"
  }"
}
```

폴더 구조

탐색기

...

▼ MODEL-SERVING-COURSE

▼ app

> __pycache__

🔗 __init__.py

🔗 auth.py

🔗 chatbot_api.py

🔗 chatbot_model.py

🔗 chatbot_schemas.py

🔗 error_handlers.py

🔗 housing_api.py

🔗 housing_model.py

🔗 housing_schemas.py

🔗 logger_config.py

🔗 main_async_solution.py

🔗 main_basic.py

🔗 main_final.py

🔗 main_params.py

🔗 main_sync_problem.py

🔗 main_v2.py

🔗 main.py

🔗 middleware.py

🔗 model_utils.py

🔗 schemas.py

🔗 sentiment_api.py

🔗 sentiment_model.py

🔗 sentiment_schemas.py

> data

▼ frontend

🔗 app_chatbot.py

🔗 app_dashboard.py

🔗 app_hello.py

🔗 app_housing.py

탐색기

...

▼ MODEL-SERVING-COURSE

▼ app

> __pycache__

🔗 __init__.py

🔗 auth.py

🔗 chatbot_api.py

🔗 chatbot_model.py

🔗 chatbot_schemas.py

🔗 error_handlers.py

🔗 housing_api.py

🔗 housing_model.py

🔗 housing_schemas.py

🔗 logger_config.py

🔗 main_async_solution.py

🔗 main_basic.py

🔗 main_final.py

🔗 main_params.py

🔗 main_sync_problem.py

🔗 main_v2.py

🔗 main.py

🔗 middleware.py

🔗 model_utils.py

🔗 schemas.py

🔗 sentiment_api.py

🔗 sentiment_model.py

🔗 sentiment_schemas.py

> data

▼ frontend

🔗 app_chatbot.py

🔗 app_dashboard.py

🔗 app_hello.py

🔗 app_housing.py

1. README.md

한국어 리뷰 감정분석 서비스

1. 프로젝트 소개

본 프로젝트는 한국어 영화 및 상품 후기를 입력하면 감정을 분석하여 **긍정 / 부정 결과와 신뢰도(confidence)**를 반환하는 서비스입니다.

FastAPI 기반의 백엔드와 Streamlit 기반의 프론트엔드를 결합하여 사용자 친화적인 AI 서비스를 구현하였습니다.

2. 선택한 태스크 및 도메인

- * 도메인: 텍스트 처리 (NLP)
- * 태스크: 감정 분석 (Text Classification)

사용자가 입력한 리뷰를 기반으로 감정을 분류하는 서비스를 구현하였습니다.

3. 사용 모델

- * 모델: `daekeun-ml/koelectra-small-v3-nsmc`

- * 특징:

- * 한국어 감정 분석 특화 모델
- * 가벼운 모델 크기로 CPU 환경에서도 빠른 추론 가능
- * Hugging Face `pipeline()`으로 간단하게 사용 가능

4. 시스템 구조

백엔드 (FastAPI)

- * `/predict` 엔드포인트 구현
- * Pydantic 기반 입력 검증
- * `run\_in\_executor`를 통한 비동기 추론
- * API Key 인증 (`Depends(verify\_api\_key)`)

프론트엔드 (Streamlit)

- * 사용자 입력 UI 제공
- * API 호출 후 결과 시각화

5. 주요 기능

- * 리뷰 입력 → 감정 분석 수행
- * 결과: 긍정/부정 + 신뢰도 출력
- * API Key 인증 적용 (401 처리)
- * 잘못된 입력에 대한 에러 처리 (422 반환)

6. 실행 방법

1. 서버 실행

```
```bash
uvicorn app.sentiment_api:app --host 0.0.0.0 --port 8000 --reload
```
```

2. 프론트 실행

```
```bash
```

```
streamlit run frontend/app_sentiment.py --server.port 8501
```

```
'''
```

```

```

## ## 7. 실행 결과

### ### Swagger 테스트

\* 정상 요청: 200 OK 반환

\* 인증 실패: 401 반환

### ### Streamlit

\* 사용자 입력 → 결과 출력 정상 동작

```

```

## ## 8. 결론

본 프로젝트를 통해 FastAPI 기반 모델 서빙과 API 인증, 프론트엔드 연동까지 하나의 완성된 AI 서비스를 구현할 수 있었다.

## 2. 발표

안녕하세요. 한국어 리뷰 감정분석 서비스를 구현한 김하늘입니다.

이번 프로젝트에서는 텍스트 데이터를 기반으로 감정을 분석하는 서비스를 만들었습니다.

제가 선택한 태스크는 감정 분석이며, 사용자가 영화나 상품 후기를 입력하면 해당 문장이 긍정인지 부정인지 분류하고 신뢰도를 함께 반환하는 서비스입니다.

사용한 모델은 Hugging Face의 koelectra 기반 감정분석 모델입니다. 이 모델을 선택한 이유는 한국어에 특화되어 있고, 모델 크기가 작아 CPU 환경에서도 빠르게 실행되기 때문입니다.

백엔드는 FastAPI로 구현하였고, /predict 엔드포인트를 통해 입력을 받아 모델 추론을 수행하도록 구성했습니다. 이때 Pydantic을 사용하여 입력값 검증을 수행했고, 잘못된 입력에 대해서는 422 에러를 반환하도록 처리했습니다.

또한 인증 기능을 추가하여 API Key가 없는 요청은 401 에러를 반환하도록 구현했습니다.

프론트엔드는 Streamlit으로 구성하여 사용자가 직접 텍스트를 입력하고 결과를 확인할 수 있도록 했습니다.

구현하면서 가장 어려웠던 부분은 API 인증과 비동기 추론 부분이었고, 특히 run\_in\_executor를 통해 모델 추론을 처리하는 부분이 처음에는 이해하기 어려웠습니다.

이번 프로젝트를 통해 모델을 단순히 사용하는 것을 넘어 실제 서비스 형태로 제공하는 과정을 경험할 수 있었습니다.

감사합니다.

### 3. 회고

이번 프로젝트에서는 FastAPI와 Hugging Face 모델을 활용하여 실제로 동작하는 AI 서비스를 구현해보았다.

Day 1~7의 내용을 기반으로 프로젝트를 진행했지만, 모든 내용을 완전히 기억하고 있는 것은 아니었기 때문에 특히 API 인증 부분과 비동기 처리 부분에서 교안을 다시 참고하게 되었다.

가장 어려웠던 부분은 API Key 인증과 에러 처리 구조를 정확히 이해하는 것이었고, 또한 run\_in\_executor를 활용한 비동기 추론 처리도 처음에는 개념적으로 이해하는 데 시간이 필요했다.

하지만 이러한 과정을 통해 단순히 모델을 사용하는 것이 아니라, 실제 서비스 형태로 제공하는 흐름을 이해할 수 있게 되었다.

다음에 다시 프로젝트를 진행한다면, 코드 구조를 더 깔끔하게 정리하고, 로깅이나 예외 처리 부분을 더 체계적으로 구성해보고 싶다.

이번 프로젝트를 통해 AI 모델을 API로 배포하고, 사용자 인터페이스까지 연결하는 전체 과정을 경험할 수 있었고, 매우 의미 있는 학습이었다.

#### 4. Day 8 체크포인트 답안

Q1. 본인의 프로젝트에서 Pydantic 검증은 어떤 잘못된 입력을 막아줍니까?

→ 빈 문자열, 공백만 입력된 문자열, 길이가 너무 긴 입력을 방지합니다.

Q2. Depends(verify\_api\_key)를 제거하면 어떤 위험이 있습니까?

→ 인증 없이 누구나 API를 호출할 수 있어 서버 과부하, 비용 증가, 모델 도용 등의 문제가 발생할 수 있습니다.

Q3. run\_in\_executor를 사용한 이유는 무엇입니까?

→ 모델 추론은 CPU를 많이 사용하는 작업이므로, 이벤트 루프를 막지 않기 위해 별도의 스레드에서 실행하기 위해 사용했습니다.

Q4. Day 1~8 중 가장 많이 참고한 Day는 어디였습니까? 왜?

→ Day 6입니다. API 인증 구조와 Depends 사용 방식이 프로젝트에 직접적으로 필요했기 때문입니다.

Q5. 이 서비스를 실제로 배포하려면 추가로 무엇이 필요합니까?

→ Docker를 통한 컨테이너화, 클라우드 서버 배포, HTTPS 적용, 로깅 및 모니터링 시스템이 필요합니다.